

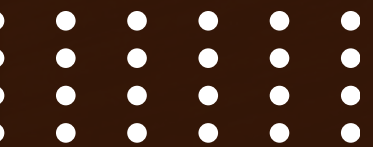
Where Every Slice is a Taste of Perfection

PIZZA SALES ANALYSIS



MYSQL

Satish Solanki





ABOUT OUR PIZZA SALES PROJECT

Our Passion for Data

In this project, I analyzed a pizza sales dataset using MySQL to uncover key business insights. The analysis involved cleaning and organizing the data, performing complex queries to identify sales trends, and deriving actionable insights such as the most popular pizza types, peak sales hours, and revenue trends. The findings can help optimize inventory management, improve marketing strategies, and boost overall sales performance.

PROBLEMS & APPROACH



Short Info About Project

This Project consists 4 tables of pizzas, pizza_types, orders, and orders_details in an Excel file. 13 different questions from basic to advanced level .



The process of setting up the database and importing tables begins with creating a new database named HizzaHuts in MySQL to store and manage pizza sales data. The schema for each table is defined in MySQL based on the data structure. The data is then imported into the respective tables in the HizzaHuts database using MySQL's import functionality. Finally, the imported data is validated to ensure accuracy and confirm that all tables are populated correctly.



**GET
READY**

SQL SKILLS USED

- Database & Table Creations
- Import Wizard
- Joins
- CTE
- ER Diagram
- Limits & Order by
- Windows Functions
- SubQuery



Questions -1 (Basic)

RETRIEVE THE TOTAL NUMBER OF ORDERS PLACED.

```
1  -- Retrieve the total number of orders placed.
2  • SELECT
3      COUNT(order_id) AS total_oreders
4  FROM
5      orders;
```

Result Grid	
	total_oreders
▶	21350

Questions -2 (Basic)

CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES.

```
1  -- Calculate the total revenue generated from pizza sales.
2  SELECT
3      ROUND(SUM(pizzas.price * order_details.quantity),
4             2) AS revenue
5  FROM
6      pizzas
7      JOIN
8      order_details ON pizzas.pizza_id = order_details.pizza_id
9
```

	revenue
▶	817860.05

Questions -3 (Basic)

IDENTIFY THE HIGHEST-PRICED PIZZA.



```
1  -- Identify the highest-priced pizza.
2  SELECT
3      pizza_id, price
4  FROM
5      pizzas
6  ORDER BY price DESC
7  LIMIT 1;
```

Result Grid			Filter Rows:
	pizza_id	price	
▶	the_greek_xxl	35.95	

Questions -4 (Basic)

IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED.

```
1  -- Identify the most common pizza size ordered.
2 • SELECT
3     pizzas.size, SUM(order_details.quantity) AS quantity
4 FROM
5     pizzas
6     JOIN
7     order_details ON pizzas.pizza_id = order_details.pizza_id
8 GROUP BY pizzas.size
9 ORDER BY quantity DESC;
```

Result Grid			Filter
	size	quantity	
▶	L	18956	
	M	15635	
	S	14403	
	XL	552	
	XXL	28	

Questions -5 (Basic)



LIST THE TOP 5 MOST ORDERED PIZZA TYPES ALONG WITH THEIR QUANTITIES.

```
-- List the top 5 most ordered pizza types along with their quantities.
• SELECT
    pizza_types.name, SUM(order_details.quantity) AS quantity
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
    JOIN
    order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.name
ORDER BY quantity DESC
LIMIT 5;
```

A screenshot of a database application window showing a 'Result Grid'. The grid has two columns: 'name' and 'quantity'. It displays the top 5 most ordered pizza types, sorted by quantity in descending order. The first row is 'The Classic Deluxe Pizza' with a quantity of 2453. The second row is 'The Barbecue Chicken Pizza' with a quantity of 2432. The third row is 'The Hawaiian Pizza' with a quantity of 2422. The fourth row is 'The Pepperoni Pizza' with a quantity of 2418. The fifth row is 'The Thai Chicken Pizza' with a quantity of 2371. The interface includes a 'Filter Rows' button and a search bar.

	name	quantity
▶	The Classic Deluxe Pizza	2453
	The Barbecue Chicken Pizza	2432
	The Hawaiian Pizza	2422
	The Pepperoni Pizza	2418
	The Thai Chicken Pizza	2371

Questions -1 (Intermediate)

JOIN THE NECESSARY TABLES TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED.

```
1  -- Join the necessary tables to find the total quantity of each pizza category ordered.
2  • SELECT
3      pizza_types.category,
4      SUM(order_details.quantity) total_orders
5  FROM
6      pizza_types
7      JOIN
8      pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
9      JOIN
10     order_details ON pizzas.pizza_id = order_details.pizza_id
11  GROUP BY pizza_types.category
```

Result Grid |   Filter Rows:

	category	total_orders
▶	Classic	14888
	Veggie	11649
	Supreme	11987
	Chicken	11050

Questions -2 (Intermediate)



DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR OF THE DAY.

```
1  -- Determine the distribution of orders by hour of the day.
2  SELECT
3      HOUR(order_time) AS hour, COUNT(order_id) AS order_count
4  FROM
5      orders
6  GROUP BY hour
7  order by order_count desc;
8
```

Result Grid			Filter R
	hour	order_count	
▶	12	2520	
	13	2455	
	18	2399	
	17	2336	
	19	2009	
	16	1920	
	20	1642	
	14	1472	
	15	1468	
	11	1231	
	21	1198	
	22	663	
	23	28	
	10	8	

Questions -3 (Intermediate)



JOIN RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS.

```
1  -- Join relevant tables to find the category-wise distribution of pizzas.
2  SELECT
3      category, COUNT(name)
4  FROM
5      pizza_types
6  GROUP BY category;
```

Result Grid			Filter Rows:
	category	count(name)	
▶	Chicken	6	
	Classic	8	
	Supreme	9	
	Veggie	9	

Questions -4 (Intermediate)



**GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE
NUMBER OF PIZZAS ORDERED PER DAY.**

```
1  -- Group the orders by date and calculate the average
2  -- number of pizzas ordered per day.
3 • SELECT
4      ROUND(AVG(total_count), 2) AS average
5  FROM
6      (SELECT
7          orders.order_date, SUM(order_details.quantity) AS total_count
8      FROM
9          orders
10     JOIN order_details ON orders.order_id = order_details.order_id
11     GROUP BY orders.order_date) AS data
12
```

Result Grid			
	average		
▶	138.47		

Questions -5 (Intermediate)

DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE.

```
1  -- Determine the top 3 most ordered pizza types based on revenue.
2 • SELECT
3     pizza_types.name,
4     SUM(pizzas.price * order_details.quantity) AS revenue
5 FROM
6     pizza_types
7     JOIN
8     pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
9     JOIN
10    order_details ON pizzas.pizza_id = order_details.pizza_id
11 GROUP BY pizza_types.name
12 ORDER BY revenue DESC
13 LIMIT 3;
```

Result Grid			Filter Rows:
	name	revenue	
▶	The Thai Chicken Pizza	43434.25	
	The Barbecue Chicken Pizza	42768	
	The California Chicken Pizza	41409.5	

Questions -1 (Advanced)



CALCULATE THE PERCENTAGE CONTRIBUTION OF EACH PIZZA TYPE TO TOTAL REVENUE.

```
1  -- Calculate the percentage contribution of each pizza type to total revenue.
2
3  ● SELECT
4      pizza_types.category,
5      ROUND((SUM(pizzas.price * order_details.quantity) / (SELECT
6          ROUND(SUM(pizzas.price * order_details.quantity),
7              2) AS revenue
8          FROM
9              pizzas
10             JOIN
11                 order_details ON pizzas.pizza_id = order_details.pizza_id)) * 100,
12          2) AS revenue
13  FROM
14      pizza_types
15      JOIN
16      pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
17      JOIN
18      order_details ON pizzas.pizza_id = order_details.pizza_id
19  GROUP BY pizza_types.category;
```

Result Grid			Filter Rows
	category	revenue	
▶	Classic	26.91	
	Veggie	23.68	
	Supreme	25.46	
	Chicken	23.96	

Questions -2 (Advanced)



ANALYZE THE CUMULATIVE REVENUE GENERATED OVER TIME.

```
1  -- Analyze the cumulative revenue generated over time.
2  • select order_date, sum(revenue) over(order by order_date) as cum_rev from
3  (SELECT
4      orders.order_date,
5      SUM(pizzas.price * order_details.quatity) AS revenue
6  FROM
7      orders
8      JOIN
9      order_details ON orders.order_id = order_details.order_id
10     JOIN
11     pizzas ON pizzas.pizza_id = order_details.pizza_id
12 GROUP BY orders.order_date) as subquery;
```

Result Grid  Filter Rows:		
	order_date	cum_rev
▶	2015-01-01	2713.85000000000004
	2015-01-02	5445.75
	2015-01-03	8108.15
	2015-01-04	9863.6
	2015-01-05	11929.55
	2015-01-06	14358.5
	2015-01-07	16560.7
	2015-01-08	19399.05
	2015-01-09	21526.4
	2015-01-10	23990.3500000000002
	2015-01-11	25862.65
	2015-01-12	27781.7
	2015-01-13	29831.3000000000003
	2015-01-14	32358.7000000000004

Questions -3 (Advanced)

DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY.

	category	name	revenue
▶	Chicken	The Thai Chicken Pizza	43434.25
	Chicken	The Barbecue Chicken Pizza	42768
	Chicken	The California Chicken Pizza	41409.5
	Classic	The Classic Deluxe Pizza	38180.5
	Classic	The Hawaiian Pizza	32273.25
	Classic	The Pepperoni Pizza	30161.75
	Supreme	The Spicy Italian Pizza	34831.25
	Supreme	The Italian Supreme Pizza	33476.75
	Supreme	The Sicilian Pizza	30940.5
	Veggie	The Four Cheese Pizza	32265.700000000065
	Veggie	The Mexicana Pizza	26780.75
	Veggie	The Five Cheese Pizza	26066.5

```
1  -- Determine the top 3 most ordered pizza types based on revenue for each pizza category
2  WITH RevenueData AS (
3      SELECT
4          pizza_types.category,
5          pizza_types.name,
6          SUM(pizzas.price * order_details.quantity) AS revenue
7      FROM
8          pizza_types
9      JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
10     JOIN order_details ON order_details.pizza_id = pizzas.pizza_id
11     GROUP BY
12         pizza_types.category, pizza_types.name
13 ),
14 RankedData AS (
15     SELECT
16         category,
17         name,
18         revenue,
19         RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
20     FROM
21         RevenueData
22 )
23 SELECT
24     category,
25     name,
26     revenue
27 FROM
28     RankedData
29 WHERE
30     rn <= 3;
```




FEEL FREE TO REACH OUT TO ME IN CASE OF ANY QUERIES



Email Address :

satish007solanki@gmail.com



Connect me :

www.linkedin.com/in/satish-solanki007



Pizza Sales Presentation

THANK YOU
FOR ATTENTION



See You Next

<https://satish-solanki.github.io/satishsolanki.github.io/>

